compare different ways of fetching and using row data with Active Record. The information below is a performance sample collected using Benchmark.memory, comparing Active Record and SQL equivalents instantiating Trip objects.

```
Comparison:
    Using select_one:       4660 allocated
   Normal Trip.first:    1988419 allocated - 426.70x more
```

Active Record allocates 426 times more objects compared with the SQL approach. The SQL approaches do not create Trip model instances so the latency involved with that could be avoided.

Wrapping Up

You now have several ways to optimize your Active Record code, starting from improved observability connecting your SQL queries to the Active Record query code that generated it.

You removed unnecessary queries from Active Record helping, conserve limited database resources.

You learned how to identify the N+1 query pattern manually, automatically, and how to fix it using Eager Loading or avoid it using Strict Loading.

You created Subqueries, Common Table Expression (CTE) queries, and Database views. You learned how Views can be Materialized to provide performance advantages. You saw how the lifecycle of versioned database views can be managed over time.

You learned about caches provided by Rails, including the Query Cache and Counter Cache.

With Active Record covered thoroughly, it's time to shift your focus away from the Rails application that generates SQL queries, and into query level observability once those queries have arrived to PostgreSQL.

In the next chapter you'll explore SQL query level performance observability and improvements. Read on to get started.